

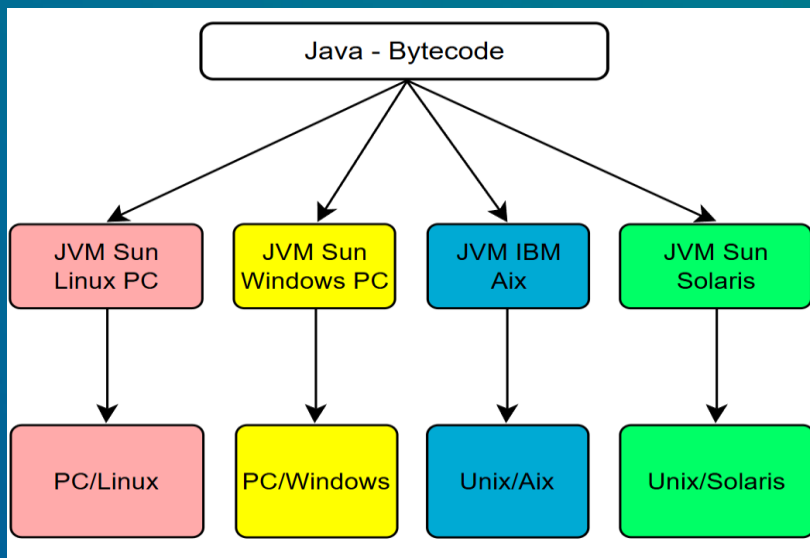


Introducción

Características de JAVA

- ❖ Independiente de Plataforma
- ❖ Orientado a objetos (POO)
- ❖ Sencillez
- ❖ Seguridad
- ❖ Arquitectura Neutral
- ❖ Portabilidad
- ❖ Robustez
- ❖ Multi-Hilo
- ❖ Interpretado
- ❖ Alto Rendimiento
- ❖ Distribuido

Independiente de Plataforma



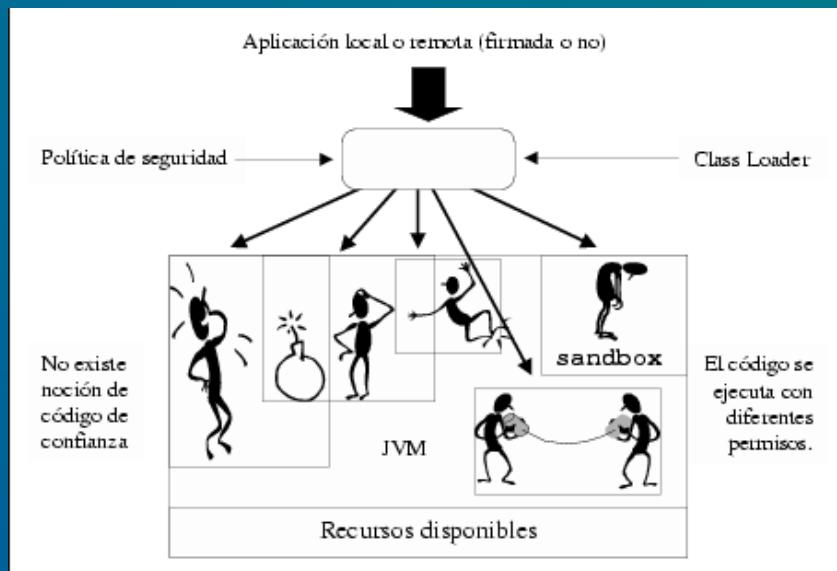
Orientado a objetos (POO)



Sencillez



Seguridad

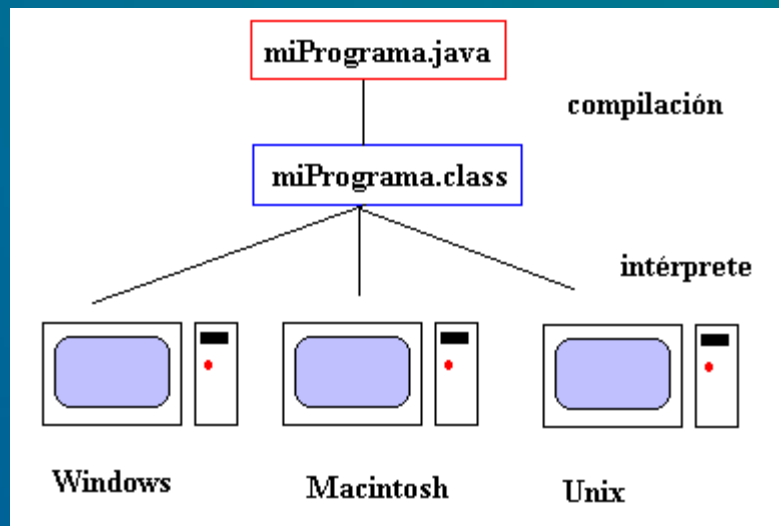


Arquitectura Neutral

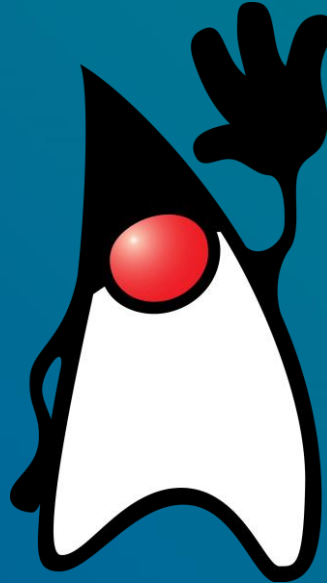


Introducción a JAVA

Portable

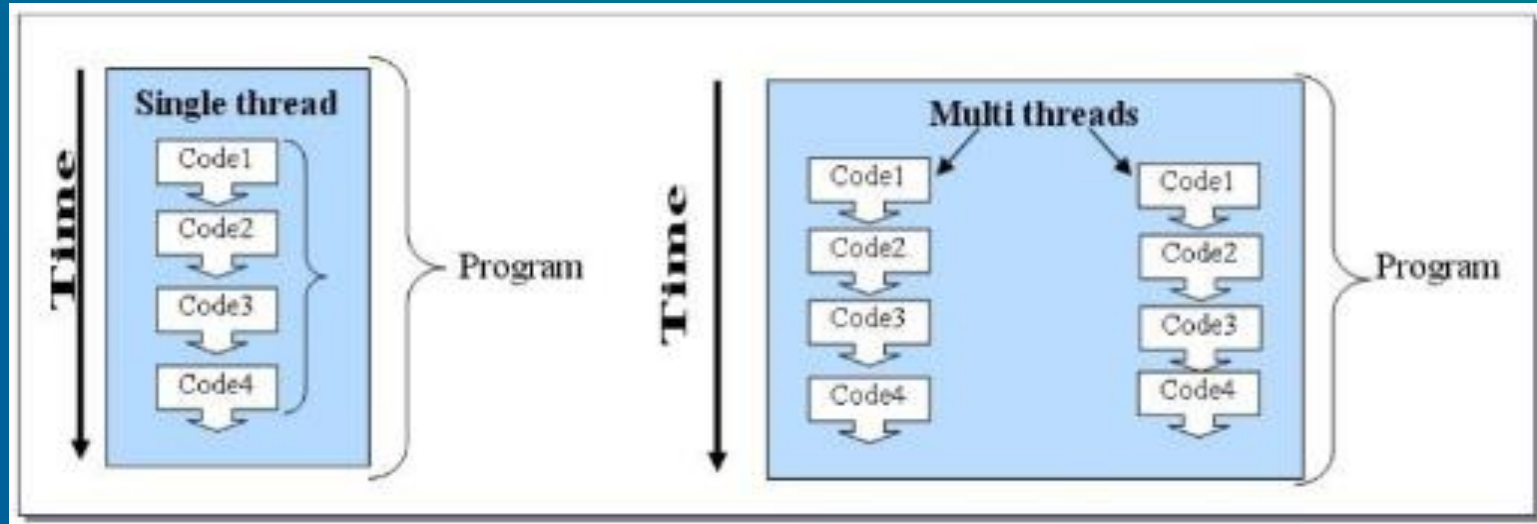


Robustez

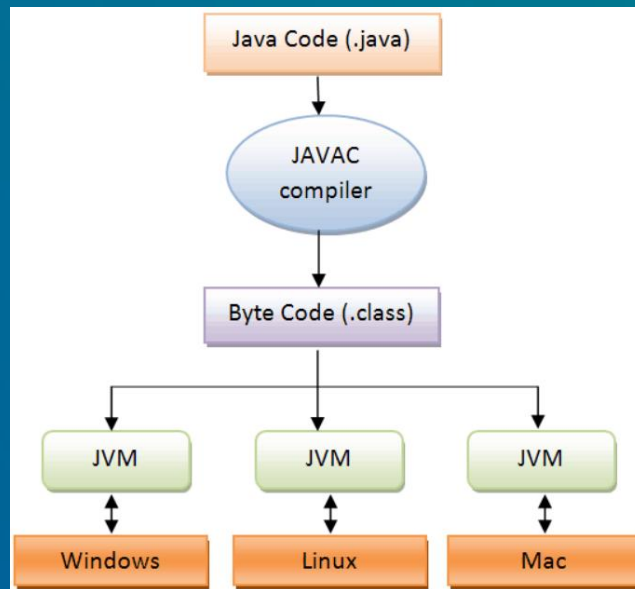


Introducción a JAVA

Multihilo



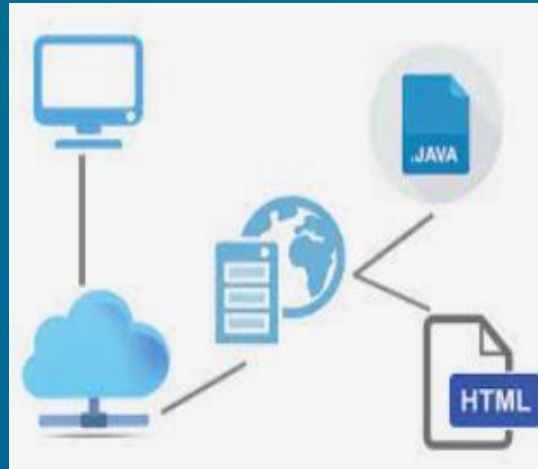
Interpretado



Alto Rendimiento



Distribuido



Instalación

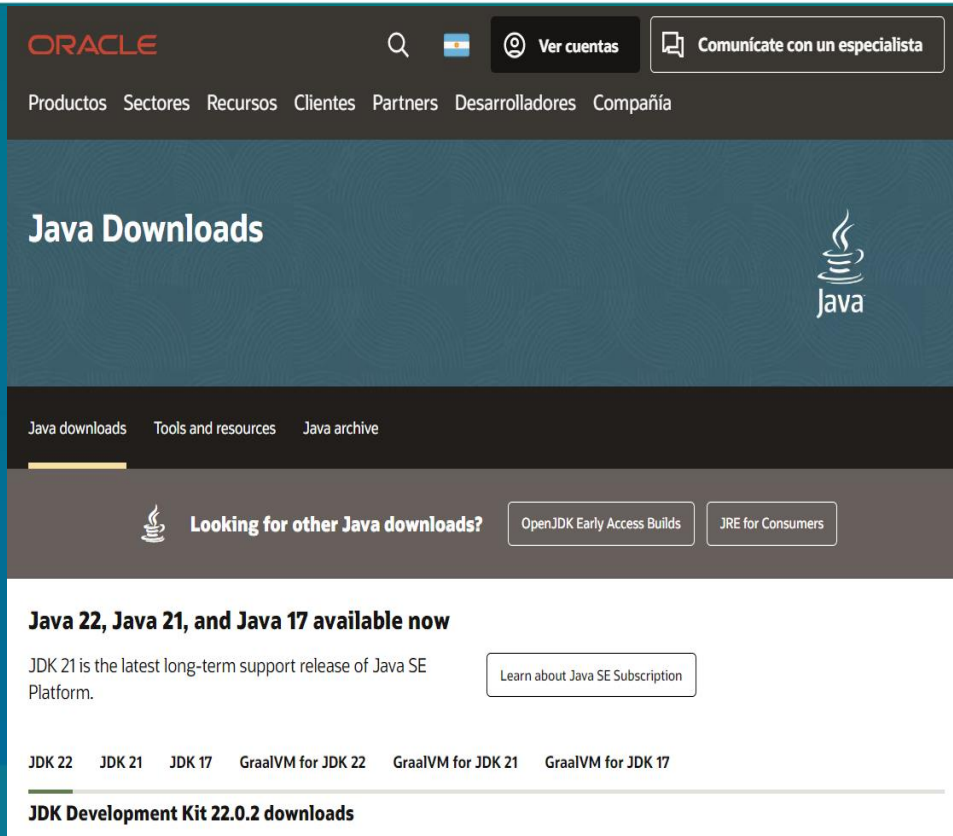
Instalar JDK, IDE y complementos.

Instalar el JDK (Java Development Kit)

Acceder a: 

Seleccionar solapa JDK 21 (LTS)

Indicar el S.O. y descargar el instalador



The screenshot shows the Oracle Java Downloads page. At the top, there's a navigation bar with the Oracle logo, a search icon, a flag for Argentina, a 'Ver cuentas' button, and a 'Comunicate con un especialista' button. Below this is a menu with links: Productos, Sectores, Recursos, Clientes, Partners, Desarrolladores, and Compañía. The main heading is 'Java Downloads' with the Java logo to the right. Underneath, there are tabs for 'Java downloads', 'Tools and resources', and 'Java archive'. A section titled 'Looking for other Java downloads?' includes buttons for 'OpenJDK Early Access Builds' and 'JRE for Consumers'. Below this, a section states 'Java 22, Java 21, and Java 17 available now' and mentions that JDK 21 is the latest long-term support release of Java SE Platform, with a link to 'Learn about Java SE Subscription'. At the bottom, there's a list of download links: JDK 22, JDK 21, JDK 17, GraalVM for JDK 22, GraalVM for JDK 21, and GraalVM for JDK 17. The final section is titled 'JDK Development Kit 22.0.2 downloads'.

Instalar el JDK (Java Development Kit)

Ejecutar el instalador y completar los pasos con el clásico :
“Siguiente, siguiente... finalizar”.



Instalar la IDE (Integrated Development Environment)



Visual Studio Code



eclipse



NetBeans

NetBeans es un entorno de desarrollo integrado (IDE) libre, gratuito y sin restricciones de uso, hecho principalmente para JAVA.

Existe además un número importante de módulos para extenderlo.

Posee, entre otros:

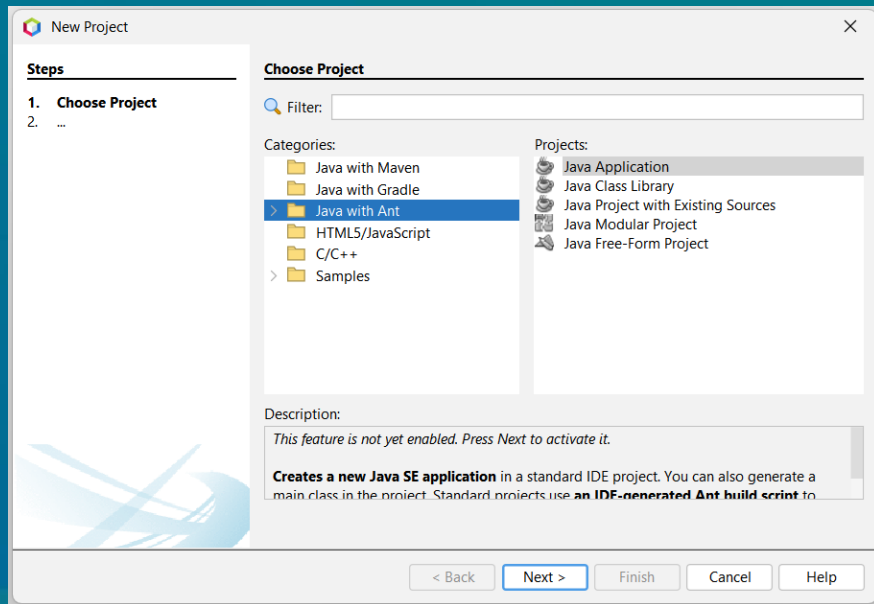
- Una terminal integrada.
- Intellisense.
- Consola de depuración.
- Gestor de módulos.
- Etc.



NetBeans

Una vez descargado el instalador, seleccionar **Nuevo proyecto**:

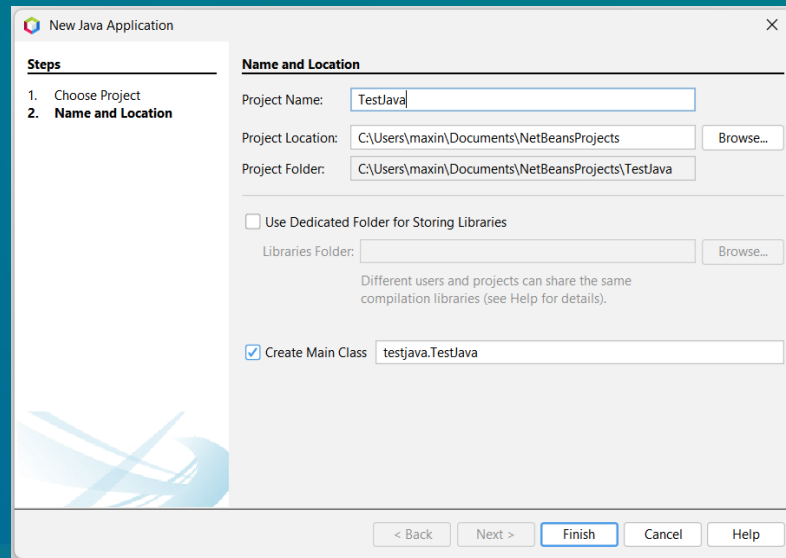
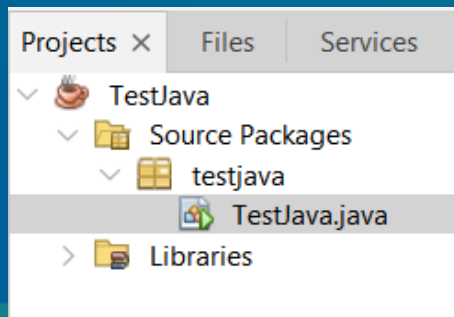
- Java with Ant -> Java Application.
- Colocar nombre del proyecto.



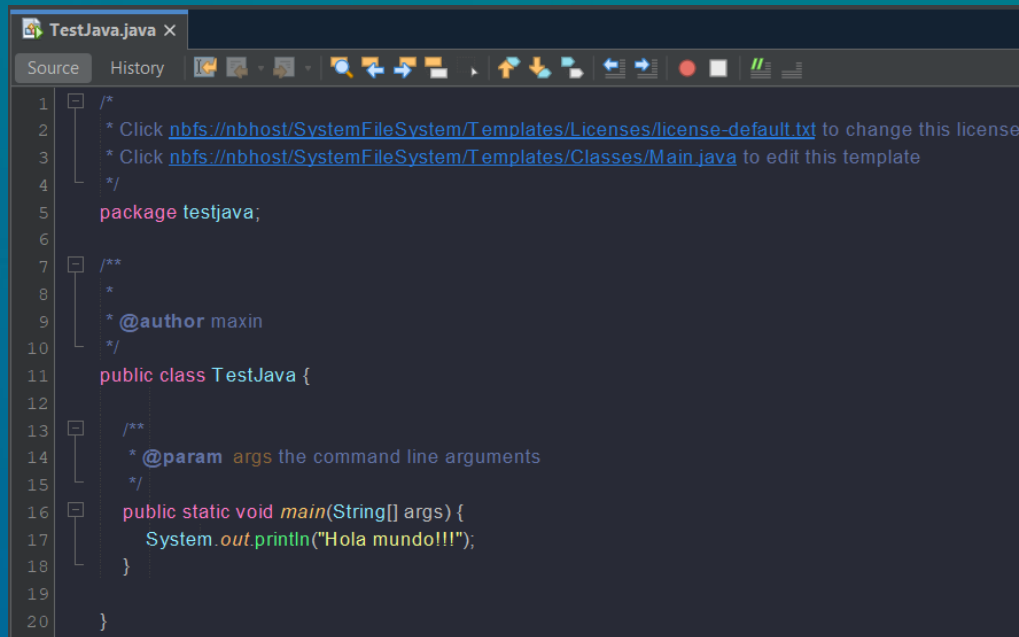
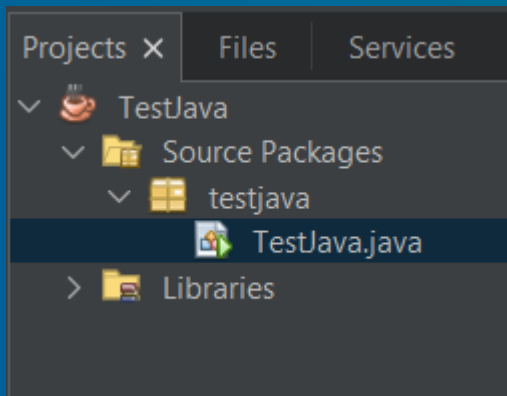
NetBeans

Una vez descargado el instalador,
seleccionar ***Nuevo proyecto:***

- Colocar nombre del proyecto.
- Finalizar
- Listo!



NetBeans



¿Cómo inicio en Java?

Sintaxis básica

Sintaxis básica

- Todo el código se escribirá en archivos con extensión *.java*.
- Los archivos contienen hasta tres estructuras de construcción:
 - Hasta una instrucción *package* seguida del nombre del paquete (siempre en la primera línea del archivo)
 - Cero o más sentencias *import* seguida del nombre cualitativo completo de las clases o *“*”* por cada paquete a importar.
 - Una definición de clase o interfaz seguida de su nombre y bloque.
- El nombre del archivo **debe** tener el mismo nombre de la clase pública o interfaz pública que contiene. Recordar que Java es *case sensitive*.

Convención de nombres

- **Clases:** Usar nombres descriptivos. Deben ser sustantivos con su primera letra en mayúscula y el resto minúscula (UpperCamelCase). Si es una palabra compuesta, las primeras letras de cada palabra en mayúsculas.
**Ejemplo: MiClase, Auto, etc.*
- **Métodos:** Usar verbos para los nombres. Comenzar en minúsculas. Si es una palabra compuesta, comenzar con mayúsculas la primera letra de la segunda palabra (camelCase).
**Ejemplo: obtenerDato, acelerar, etc.*
- **Paquetes:** Deben ser escritos en minúsculas y deben representar un nombre de dominio (similar a los namespaces de C#).

Convención de nombres

- ❖ **Variables:** Utilizar nombres cortos y descriptivos. Comenzar con minúsculas (si es compuesta, la primera letra de la segunda palabra, en mayúscula). Utilizar nombres que describen la intención de la variable.
Ejemplo: nombre, nombreCliente, edad, etc.
- ❖ **Constantes:** Similar a las variables pero todos sus caracteres en mayúsculas y si son compuestas, separadas por un guión bajo.
Ejemplo: CANTIDAD, EDAD_MAXIMA, etc.

Declaraciones

❖ Variables:

Tipo identificador;

Tipo identificador = valor;

Dónde:

Tipo: Indica el tipo de la variable.

Identificador: establece el nombre de la variable.

```
String nombre;
```

```
nombre = "Juan";
```

```
int edad = 22;
```

```
float precioProducto = 25.33;
```

Declaraciones

❖ Constantes:

final Tipo IDENTIFICADOR = valor;

```
final int VALOR_ENTERO = 55;
```

Dónde:

final: es una palabra reservada que indica se definirá una constante.

Tipo: Indica el tipo de la constante.

IDENTIFICADOR: establece el nombre de la constante.

valor: establece el valor de la constante.

Variables escalares y no escalares

- ❖ Las **variables escalares** son variables (o constantes) que contienen un dato atómico y unidimensional.
- ❖ Las **variables no escalares** son: arrays (vectores), listas y objetos, que pueden tener almacenado en su estructura más de un valor.

```
//ESCALARES  
double numero = 12.33;  
final int CONSTANTE = 6;  
  
//NO ESCALARES  
int[] numeros = new int[25];  
Padre objeto = new Padre();  
ArrayList<String> lista = new ArrayList<>();
```

Sintaxis básica

Operadores: aritméticos y lógicos

Operadores aritméticos

Asignación	=
Adición	+
Sustracción	-
Multiplicación	*
División	/

Módulo	%
Mayor que	>
Menor que	<
Mayor o igual que	>=
Menor o igual que	<=

Operadores lógicos

Igualdad	==
Desigualdad	!=
AND	&&
OR	
Negación	!

Sintaxis básica

Estructuras de control: condicionales y repetitivas

Condicionales

❖ Estructuras **if** con varios formatos:

```
if(x < 10)
    x++;

if(x > 10)
{
    x--;
    y = 0;
}
```

```
if(y == 0)
{
    x = 3;
}
else
{
    x = 0;
}
```

```
if(x == y)
{
    y--;
    x++;
}
else if(x > y)
{
    y += x;
}
else
{
    x = y;
}
```

Condicionales

❖ Estructuras **switch**:

```
switch(x)
{
    case 6:
        //código
        break;
    case 2:
        //código
        break;
    default:
        //código defecto
        break;
}
```

```
String cadena = "hola";
switch(cadena)
{
    case "hola":
        //código
        break;
    case "mundo":
        //código
        break;
    default:
        //código defecto
        break;
}
```

Repetitivas

❖ Estructuras **for**:

```
//inicialización; prueba; acción
for(int index = 0; index < 5; index++)
{
    //código
}
```

```
//tipo elemento : colección de elementos
for(iterable_type iterable_element : iterable)
{
    //código
}
```

```
int[] coleccion = new int[2];
for (int i : coleccion)
{
    //código
}
```

Repetitivas

❖ Estructuras **while** y **do-while**:

```
while(x < 10)
{
    x++;
}
```

```
do
{
    x++;
} while (x < 10);
```

Entrada y salida de datos

Entry point

Entry point

El punto de entrada a los programas de Java es el método público y estático **main**.

Dónde:

static: es el modificador que permite ejecutar un método sin tener que instanciar un objeto.

void: indica que este método no retornará ningún valor.

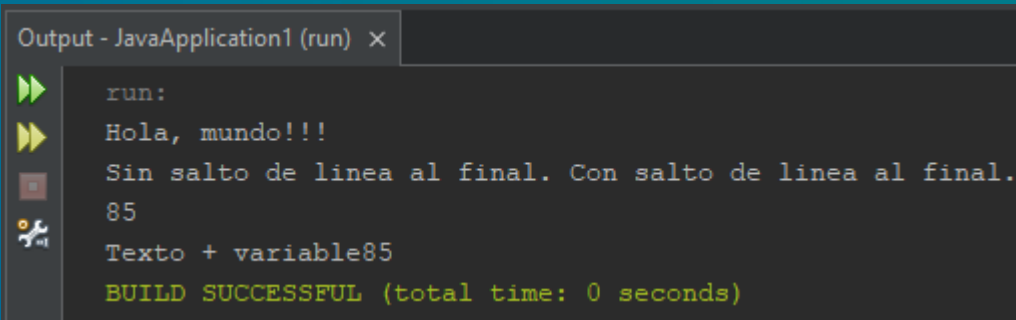
String[] args: es un array de tipo String que puede recibir el método.

```
public class App {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args)  
    {  
        System.out.println("Hola, mundo!!!");  
    }  
}
```

Salida por consola

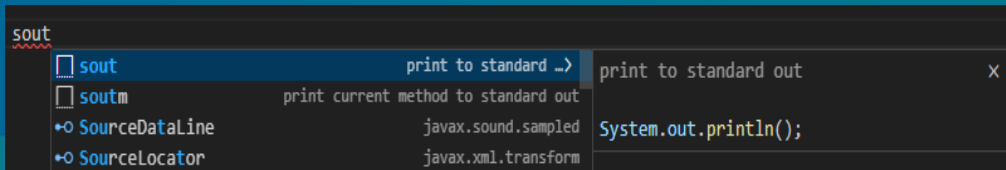
- ❖ Para imprimir por la salida estándar se utilizan los métodos estáticos:

```
public class App {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args)  
    {  
        System.out.println("Hola, mundo!!!");  
  
        int variable = 85;  
  
        System.out.print("Sin salto de linea al final. ");  
  
        System.out.println("Con salto de linea al final.");  
  
        System.out.println(variable);  
  
        System.out.println("Texto + variable" + variable);  
    }  
}
```



```
Output - JavaApplication1 (run) x  
  
run:  
Hola, mundo!!!  
Sin salto de linea al final. Con salto de linea al final.  
85  
Texto + variable85  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Utilizar fragmentos de código:



```
sout  
[ ] sout print to standard ...> print to standard out x  
[ ] soutm print current method to standard out  
+o SourceDataLine javax.sound.sampled  
+o SourceLocator javax.xml.transform  
System.out.println();
```

Ingreso por consola

- ❖ Para ingresar desde la consola se utilizan los métodos de instancia:

```
//importar el paquete java.util.Scanner;  
Scanner input = new Scanner(System.in);  
  
String cadena = input.nextLine();  
Double doble = input.nextDouble();  
Integer entero = input.nextInt();  
  
//NO olvidar cerrar el input  
input.close();
```

Utilizando try con recursos:

```
String cadena;  
Double doble;  
Integer entero;  
try ( Scanner input = new Scanner(System.in) )  
{  
    cadena = input.nextLine();  
    doble = input.nextDouble();  
    entero = input.nextInt();  
    //NO olvidar cerrar el input  
}  
catch (Exception e)  
{  
    System.err.println(e);  
}
```

Ejercitación